Step 7 - Delta Lake Deep Dive

Phase B · Time Travel · MERGE · SCD Type 2 · ~20 min

> **Goal:** Delta deep dive on YOUR tables: DESCRIBE HISTORY, time travel, idempotent MERGE upsert, SCD Type 2 (the Toronto-to-Mississauga move), OPTIMIZE/Z-ORDER, VACUUM dry-run. Artifact: 06_delta_lab.ipynb

LEARN

1. Every Delta write is a version recorded in _delta_log/. DESCRIBE HISTORY is the audit ledger: who, when, what operation.

2. Time travel: VERSION AS OF / TIMESTAMP AS OF reads the table as it was. OSFI asks 'state on March 15?' - that is one SELECT, not a backup restore.

3. MERGE = exactly-once loads. whenMatchedUpdateAll + whenNotMatchedInsertAll guarantees no duplicate transaction_id when sources resend. Rerun-safe at 3 AM.

4. SCD Type 2: expire the old row (is_current=false, end_date=today), insert the new current row. History preserved - OSFI lineage needs the address on file AT THE TIME of any transaction.

LAB — notebook 06_delta_lab (Serverless)

Cell 1 — Setup:

```
import pyspark.sql.functions as F
from delta.tables import DeltaTable
VOL = "/Volumes/workspace/default/maplebank"
SILVER, GOLD = f"{VOL}/silver", f"{VOL}/gold"
TXN = f"{SILVER}/transactions"
```

Cell 2 — Read the ledger:

```
spark.sql(f"DESCRIBE HISTORY delta.`{TXN}`") \
     .select("version","timestamp","operation").show(10, truncate=False)
```

Cell 3 — Time travel:

```
late = spark.read.format("delta").load(TXN).limit(5) \
    .withColumn("transaction_id", F.concat(F.lit("LATE_"), F.col("transaction_id")))
late.write.format("delta").mode("append").save(TXN)

current = spark.read.format("delta").load(TXN).count()
v0 = spark.read.format("delta").option("versionAsOf", 0).load(TXN).count()
print(f"v0={v0:,}  current={current:,}  diff={current-v0}")
```

Cell 4 — MERGE upsert (run twice - count stays the same = idempotent):

```
existing_id = spark.read.format("delta").load(TXN) \
    .filter(~F.col("transaction_id").startswith("LATE_")) \
    .select("transaction_id").first()[0]

resent = spark.read.format("delta").load(TXN) \
    .filter(F.col("transaction_id") == existing_id) \
    .withColumn("amount_cad", F.lit(7777.77))
brand_new = resent.withColumn("transaction_id", F.lit("TXN_NEW_001")) \
                  .withColumn("amount_cad", F.lit(123.45))
incoming = resent.union(brand_new)

DeltaTable.forPath(spark, TXN).alias("t") \
    .merge(incoming.alias("s"), "t.transaction_id = s.transaction_id") \
    .whenMatchedUpdateAll() \
    .whenNotMatchedInsertAll() \
    .execute()
```

Cell 5 — SCD Type 2 (the move):

```
SCD2 = f"{GOLD}/dim_customer_scd2"

# 5a. initialize: all current
spark.read.format("delta").load(f"{SILVER}/customers") \
    .select("customer_id","first_name","last_name","city","province","postal_code_fsa") \
    .withColumn("is_current", F.lit(True)) \
    .withColumn("start_date", F.current_date()) \
    .withColumn("end_date",   F.lit(None).cast("date")) \
    .write.format("delta").mode("overwrite").save(SCD2)

# 5b. change event
mover_id = spark.read.format("delta").load(SCD2) \
    .filter(F.col("city") == "Toronto").select("customer_id").first()[0]
update = spark.read.format("delta").load(SCD2) \
    .filter(F.col("customer_id") == mover_id) \
    .withColumn("city", F.lit("Mississauga")) \
    .withColumn("postal_code_fsa", F.lit("L5B"))

# 5c. expire the old current row
DeltaTable.forPath(spark, SCD2).alias("t") \
    .merge(update.alias("s"),
           "t.customer_id = s.customer_id AND t.is_current = true") \
    .whenMatchedUpdate(condition="t.city <> s.city",
        set={"is_current": "false", "end_date": "current_date()"}) \
    .execute()

# 5d. insert the new current row
update.withColumn("is_current", F.lit(True)) \
      .withColumn("start_date", F.current_date()) \
      .withColumn("end_date",   F.lit(None).cast("date")) \
      .write.format("delta").mode("append").save(SCD2)

# 5e. proof: TWO rows for one customer
spark.read.format("delta").load(SCD2) \
    .filter(F.col("customer_id") == mover_id) \
    .select("customer_id","city","is_current","start_date","end_date") \
    .show(truncate=False)
```

Cell 6 — OPTIMIZE + VACUUM dry-run:

```
spark.sql(f"OPTIMIZE delta.`{TXN}` ZORDER BY (customer_id)")
spark.sql(f"VACUUM delta.`{TXN}` DRY RUN").show(5, truncate=False)
# NEVER real VACUUM in this lab - it erases time-travel history.
# Banking rule: VACUUM retention is a compliance decision, not engineering.
```

COMMIT + CHECKPOINT

Export to databricks/. Pass criteria: v0 vs current differ by 5; MERGE before/after differs by exactly 1 and reruns are stable; SCD2 query shows the Toronto row (is_current=false, end-dated) and the Mississauga row (is_current=true). Phase B complete - a working lakehouse with audit history.